

day01

一、项目创建

名称	修改日期	类型	大小
.idea PyCharm的配置文件夹（软件需要依赖一些其他的東西，会自动进行配置） --- 自动生成的	2026/3/30 10:10	文件夹	
.venv 虚拟环境目录 --- 安装的扩展（库，依赖包），都默认放在这个目录的，为了方便统一进行管理	2026/3/30 10:10	文件夹	
main.py 主模块	2026/3/30 10:10	PY 文件	1

二、测试题

1、Craps游戏

要求

- 玩家摇两颗骰子，如果第一次摇出7点或者11点，玩家胜
- 如果第一次摇出了2点、3点或者12点，庄家胜，其他情况游戏继续
- 玩家重新摇骰子，如果摇出了7点，庄家胜，如果摇出了第一次的点数，玩家胜
- 如果没有分出胜负，游戏继续

```
"""
Craps游戏
要求
玩家摇两颗骰子，如果第一次摇出7点或者11点，玩家胜
如果第一次摇出了2点、3点或者12点，庄家胜，其他情况游戏继续
玩家重新摇骰子，如果摇出了7点，庄家胜，如果摇出了第一次的点数，玩家胜
如果没有分出胜负，游戏继续

两颗骰子最大点数：12，一颗是6
"""

# 模拟骰子 使用random库
# import random

# randint() 不遵循左闭右开的特点 --- range() 需要遵循左闭右开的特点
# first_point = random.randint(1, 6) + random.randint(1, 6)
# print(f'玩家第一次摇出了 {first_point}点。')

# if first_point == 7 or first_point == 11:
#     print('玩家胜')
# elif first_point == 2 or first_point == 3 or first_point == 12:
#     print('庄家胜')

# if first_point in [7, 11]:
#     print('玩家胜')
```

```
# elif first_point in [1, 3, 12]:
#     print('庄家胜')

"""
Craps游戏
要求
玩家摇两颗骰子，如果第一次摇出7点或者11点，玩家胜
如果第一次摇出了2点、3点或者12点，庄家胜，其他情况游戏继续
玩家重新摇骰子，如果摇出了7点，庄家胜，如果摇出了第一次的点数，玩家胜
如果没有分出胜负，游戏继续

plus版本
1、总金额设置
2、可以下注
3、总金额可以进行减少或者增多，给出对应的提示信息

两颗骰子最大点数：12，一颗是6

循环嵌套
"""
# 模拟骰子 使用random库
import random
first_point = random.randint(1, 6) + random.randint(1, 6)
print(f'玩家第一次摇出了 {first_point}点.')
if first_point in (7, 11):
    print('玩家胜')
elif first_point in (2, 3, 12):
    print('庄家胜')
else:
    # 其他情况游戏继续
    while True:
        current_point = random.randint(1, 6) + random.randint(1, 6)
        print(f'玩家摇出了 {current_point}点.')
        if current_point == 7:
            print('庄家胜')
            break
        elif current_point == first_point:
            print('玩家胜')
            break
```

2、craps升级版本

```
# 使用random库
import random

# 添加金额
money = 2000

# 不确定玩多少次（没有钱了，就结束点），使用while循环
while money > 0:
    print(f'玩家的资产是: {money}')
```

```

# 要排除所有的其他因素，后续代码才能执行
while True:
    # 下注操作
    debt = int(input('请下注:'))
    if debt <= 0:
        print('下注必须大于0')
    elif debt > money:
        print(f'下注金额不能超过当前的资产: {money}')
    else:
        break

first_point = random.randint(1, 6) + random.randint(1, 6)
print(f'玩家第一次摇出了 {first_point}点.')
if first_point in (7, 11):
    print('玩家胜')
    money += debt
elif first_point in (2, 3, 12):
    print('庄家胜')
    money -= debt
else:
    # 其他情况游戏继续
    while True:
        current_point = random.randint(1, 6) + random.randint(1, 6)
        print(f'玩家摇出了 {current_point}点.')
        if current_point == 7:
            print('庄家胜')
            money -= debt
            break
        elif current_point == first_point:
            print('玩家胜')
            money += debt
            break

print('游戏已结束，您已经破产! ')

```

三、基础回顾练习题

1、字符串清洗与格式化

要求： 输入字符串 " User: JOHN_Smith; RegDate: 2023-05-12; "，完成：

- 移除首尾空格及多余分号
- 将用户名转换为首字母大写 (John_smith)
- 将日期格式改为 12/05/2023
- 输出： User: John_smith; RegDate: 12-05-2023

```

text = " User: JOHN_Smith; RegDate: 2023-05-12; "
print(text)
print('-' * 50)
# 切片操作
# 切割（split） 把字符串切割成列表，根据指定字符进行切割

```

```
# capitalize() 首字母大写
step1 = text.strip()[::-1].split('; ')[0].split(':')[1].capitalize()
print(step1)

# replace() 替换
# join() 前面表示把列表转成字符串后，需要使用什么字符来拼接。参数是列表
step2 = '-'.join(text.strip()[::-1].split('; ')[1].split(':')[1].replace('-',
'/').split('/')[::-1])
print(step2)

# 输出结果
result = f'User: {step1}; RegDate: {step2}'
print(result)
```

2、双色球随机选号

要求

- 6个红色球 + 1个蓝色球
- 红色球：号码为 1 - 33，摇出6个球
- 蓝色球：号码为 1 - 16，摇出1个球

效果图

请输入要打印的注数:5

```
12, 15, 19, 22, 26, 31 | 06
03, 10, 21, 22, 27, 32 | 03
02, 04, 10, 12, 25, 32 | 03
02, 11, 18, 26, 27, 29 | 11
06, 08, 30, 31, 32, 33 | 03
```

初始版本

```
import random
# [x for x in range(1, 34)] 列表推导式写法（简化方式）
# 红色球
red_balls = [x for x in range(1, 34)]
# random.sample(red_balls, 6) 可以从列表中选出指定数量的元素
select_balls = random.sample(red_balls, 6)

# 蓝色球
blue_balls = random.randint(1, 16)

# 组合（红色球 + 蓝色球）
```

```
select_balls.append(blue_balls)

print(select_balls)
```

plus版本

```
import random

n = int(input('请输入要打印的注数:'))

# 定义函数来做这个事情，因为函数可以传递参数，也可以进行代码复用
# n 表示的是打印几注
def create_balls(n):
    for _ in range(n):
        # 红色球
        red_balls = [x for x in range(1, 34)]
        # random.sample(red_balls, 6) 可以从列表中选出指定数量的元素
        select_balls = random.sample(red_balls, 6)
        # 进行排序操作
        # reverse=False 表示升序（不写是一样的效果）
        # reverse=True 表示降序
        select_balls.sort()

        # 蓝色球
        blue_balls = random.randint(1, 16)

        # ', '.join()
        # 表示格式化红色球
        fm_red = ', '.join([f'{val:02d}' for val in select_balls])

        # 表示格式化蓝色球
        fm_blue = f'{blue_balls:02d}'

        # 输出最终结果
        print(f'{fm_red} | {fm_blue}')

create_balls(n)
```

四、切片操作

- 1、切片指的是把容器类型从容器里面取出来，在取的过程中，可以规定从什么位置开始、从什么位置结束、也可以指定步长
- 2、切片使用 `[]` 来进行操作
- 3、语法

```
# 0123
t = 'he!a!o, 美女帅哥! '
# 索引（下标、index），默认是从0开始的
# 取的操作，正向索引（从左往右），反向索引（从右到左）
```

```

print(t[0])
print(t[1])
print(t[-1])
print(t[-2])
print(t[-3])

print('-' * 20)
# 左闭右开特点（包含左边的，不包含右边的结束位置）
print(t[0:3])
# 表示从1开始，取完
print(t[1:])
print(t[:3])

print('-' * 20)

print(t[0:5:2])

print('-' * 20)

# 注意点：如果使用负数来进行切片操作，那么需要注意的是，索引默认是正向的，需要修改为负向索引
print(t[-1:-5:-1])

print('-' * 20)

lis = ['法外狂徒张三', '坤坤', '鸡太美', '篮球', 'rap', '唱跳']
print(lis[0])
print(lis[-1])
print(lis[1:3])

# 反转字符串或者列表
print(lis[::-1])

```

五、作业

1、从“城市-区域”字段中拆分两级信息

数据列：`locations = ["北京市-海淀区", "上海市-浦东新区", "广州市-天河区"]`

拆分为两个列表：

→ `cities = ["北京市", "上海市", "广州市"]`

→ `districts = ["海淀区", "浦东新区", "天河区"]`

2、统计字符串的个数

```
text = 'abcdffqdafg'
```

```
result = {  
  a: 2,  
  b: 1  
}
```